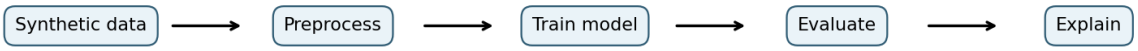
DAY 37

Model Interpretability & Explainable AI

A practical guide to understanding why machine learning models make predictions



Project: Explainable AI Student Performance Prediction using synthetic data.

Learning goal: Train a basic model, evaluate it, inspect global feature importance, and explore an individual prediction while understanding the limits of each explanation.

Responsible-use note: The project uses fictional synthetic data for education. It must not be used to make real student admissions, grading, discipline, or other high-impact decisions.

1. Learning objectives

• Explain model interpretability and Explainable AI (XAI).

• Distinguish global explanations from local explanations.

• Understand feature importance and its limitations.

• Explain the role of preprocessing pipelines and model evaluation.

• Describe the basic purpose of SHAP and LIME.

• Interpret model outputs without treating explanations as proof of causation.

2. What is model interpretability?

Model interpretability is the degree to which a person can understand how a model behaves or how it reaches a prediction. A simple decision tree may be easier to inspect than a large ensemble or deep neural network. Explainable AI (XAI) refers to methods that help people inspect, understand, and communicate model behavior.

Why does it matter?

• Trust and communication: people need a meaningful account of model behavior.

• Debugging: explanations can reveal reliance on irrelevant or suspicious inputs.

• Fairness review: explanations may help flag concerning patterns for further investigation.

• Monitoring: explanations can change when data distributions or model behavior change.

An explanation does not automatically make a model accurate, fair, safe, or causally valid. It is one part of a broader review process.

3. Interpretable models and black-box models

Model type	Typical interpretability	Example
Linear model	Often easier to inspect when inputs and assumptions are understood	Logistic regression coefficients
Small decision tree	Can be visualized as a sequence of splits	If/then rules
Random forest	Harder to explain as one simple set of rules	Many trees combined
Deep neural network	Often complex to inspect directly	Many learned layers

The boundary is not absolute. A linear model can still be difficult to explain if its features are highly transformed, correlated, or poorly documented. A complex model may also have useful post-hoc explanation tools.

4. Global versus local explanations

Global explanation

A global explanation describes patterns across many predictions. Examples include permutation feature importance or a summary of how the model behaves across a dataset.

Local explanation

A local explanation focuses on one prediction, such as why the model predicted that one fictional student's outcome was likely to be in a particular class.

```
# Global question:
# Which input features matter most to model performance overall?

# Local question:
# For this one example, how does changing an input affect the model output?
```

These questions are different. A feature that matters globally may have little effect on a particular example. A local explanation does not necessarily generalize to all students or other datasets.

5. Feature importance

Feature importance attempts to estimate how much each input feature contributes to a model's behavior or predictive performance. The exact meaning depends on the method.

Permutation feature importance

Permutation importance measures how much a chosen evaluation score changes when one feature's values are randomly shuffled. If shuffling a feature reduces model performance, the model may rely on information in that feature. Calculate it on held-out data when possible.

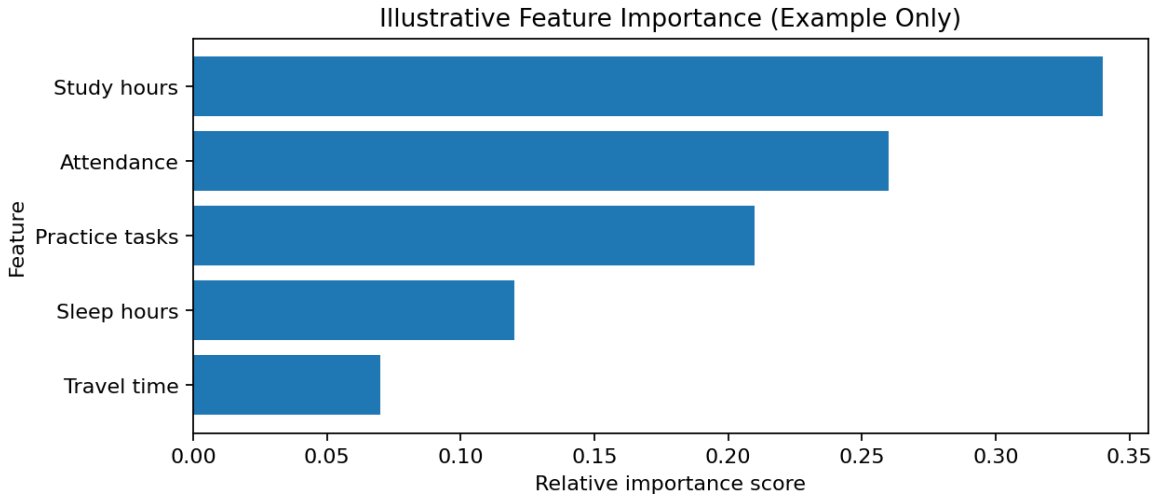
```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    trained_model,
    X_test,
    y_test,
    n_repeats=10,
    random_state=42,
    scoring="f1"
)

print(result.importances_mean)
```

Important limitation: correlated features can share predictive information. Shuffling one correlated feature may make its importance appear low because another feature provides similar information. Importance is not proof of causation or moral significance.

6. Illustrative feature-importance chart



The chart above is illustrative only, not a result from a real dataset or the companion model. In a real analysis, show the actual metric, dataset split, method, uncertainty across repeats, and limitations.

7. SHAP and LIME: basic idea

SHAP

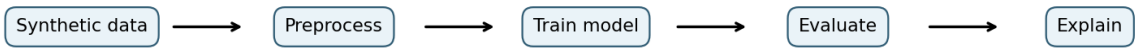
SHAP is a family of methods that assigns feature-attribution values to explain a prediction relative to a reference or baseline. SHAP can support local explanations and summary views across many examples. The meaning depends on the model, background data, feature dependencies, and explainer configuration.

LIME

LIME explains a particular prediction by creating perturbed examples near the instance and fitting a simpler model that approximates the original model locally. It can be intuitive, but results may vary with perturbation choices and settings.

Beginner advice: Learn global and local explanation concepts first. Add SHAP or LIME when you understand the model type, data representation, and assumptions. Do not claim the companion project uses SHAP or LIME unless those tools are actually implemented.

8. Project workflow



- Create synthetic records with fictional study-related inputs and a target label.
- Split the data into training and test sets before model evaluation.
- Use a preprocessing pipeline for numeric and categorical columns.
- Train a classification model on training data only.
- Evaluate held-out predictions using suitable metrics.
- Inspect permutation feature importance on the test set.
- Examine a fictional example and describe how changes in one feature affect the model output.

9. Data preparation and pipelines

A pipeline makes preprocessing and prediction steps part of one consistent workflow. Numeric features may need missing-value imputation and scaling. Categorical features may need imputation and one-hot encoding. Fit transformations on training data only to avoid data leakage.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_columns),
    ("cat", categorical_pipeline, categorical_columns)
])

model = Pipeline([
    ("preprocess", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

This is a teaching example. In the complete .py project file, feature names and dataset creation are defined in code. A pipeline helps keep training and prediction preprocessing aligned.

10. Model evaluation before explanation

An explanation cannot compensate for a poorly evaluated model. Use a held-out test set and choose metrics appropriate to the task. For classification, accuracy, precision, recall, F1, and a confusion matrix can provide complementary views. If

classes are imbalanced, accuracy alone can be misleading.

```
from sklearn.metrics import classification_report, confusion_matrix

predictions = model.predict(X_test)
print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

Do not report made-up model scores. Run the companion script to see results from its generated synthetic dataset, and remember that results on synthetic data do not establish real-world performance.

11. Explaining one prediction responsibly

The companion script includes a local sensitivity-style explanation: it changes one input feature at a time for a fictional example and observes how the model output changes. This can help demonstrate model behavior, but it is not a formal SHAP or LIME explanation and does not prove causality.

```
# Conceptual illustration only
original_probability = model.predict_proba(example)[0, 1]

# Change one feature in a copy of the example,
# run the prediction again, and compare the probabilities.
# Keep other features fixed for this simple sensitivity check.
```

When explaining one prediction, state which class the model predicted, what output or probability was inspected, which feature was varied, and what was held constant. Mention that other feature combinations can produce different behavior.

12. Common mistakes

Mistake	Why it is a problem
Treating feature importance as causation	Predictive association does not prove that changing a feature causes the outcome.
Explaining a model before evaluating it	A detailed explanation of an unreliable model can still mislead.
Using test data during preprocessing fit	This can leak information and inflate evaluation results.
Calling every explanation SHAP	Permutation importance and sensitivity checks are different techniques.
Assuming one local explanation applies to everyone	Individual predictions can depend on different feature combinations.
Using synthetic results as real evidence	Synthetic patterns may not match real people or real educational systems.
Ignoring privacy and fairness	Student data are sensitive and educational predictions can affect opportunities.

13. Practice tasks

Task 1: Definitions

Explain interpretability in your own words and give one reason XAI is useful.

Task 2: Global or local?

Classify a feature-importance chart for the full test set and an explanation for one prediction.

Task 3: Permutation importance

Explain what it might mean if shuffling a feature reduces F1 substantially.

Task 4: Limits

Why does a high feature-importance score not prove causation?

Task 5: Evaluation

Why should you evaluate a model before explaining its decisions?

Task 6: Responsible use

Name two reasons synthetic student data should not be used to make real academic decisions.

14. Practice task solutions

Solution 1

Interpretability is how well people can understand a model's behavior. XAI can support debugging, communication, and investigation of suspicious patterns.

Solution 2

A summary chart across a test set is global. An explanation focused on one prediction is local.

Solution 3

The model may rely on information in that feature for the selected score. Check uncertainty, correlated features, data leakage, and whether the finding is stable.

Solution 4

Importance measures predictive reliance or contribution under a method's assumptions. It does not establish what would happen under a real-world intervention.

Solution 5

If a model performs poorly or fails for a subgroup, an explanation alone does not make its predictions useful. Evaluation establishes a baseline for interpreting model behavior.

Solution 6

Synthetic data may not reflect real student experiences, and the model may encode arbitrary patterns. Real educational decisions also require privacy, fairness, oversight, and validation.

15. Quick revision checklist

- ■ I can define model interpretability and XAI.
- ■ I can distinguish global from local explanations.
- ■ I understand what permutation feature importance measures.
- ■ I can describe SHAP and LIME at a high level.
- ■ I know an explanation does not prove causation.
- ■ I understand why data leakage can distort evaluation.
- ■ I can explain why synthetic student data are only for learning.

16. Final takeaway

Explainable AI helps us inspect and communicate model behavior. The strongest workflow combines sound data preparation, honest evaluation, suitable explanation methods, and clear limitations. Use explanations to ask better questions about a model, not as unquestionable proof that its decisions are correct.

Companion project file: Day-37-Explainable-AI-Student-Performance-Project.py. It is syntax-checked but has not been verified through a full end-to-end runtime test in this step.